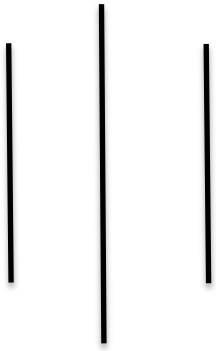
Tribhuvan University

Faculty of Humanities and Social Science

SWASTIK COLLEGE



Course: Mobile Programming(CACS351)

Title : Contact Management Android Application (V1)

Submitted By:

Anuradha Dulal (113102068)

BCA 6th Semester

Submitted To:

Sujan Poudel

Lecturer, BCA Department

Swastik College, Bhaktapur

April, 2026

ABSTRACT

The Contact Management Android Application is an Android-based system developed to simplify the storage and organization of personal and professional contacts. The application is built using Java in Android Studio and utilizes SQLite as a local database for efficient and secure data handling.

The system is divided into multiple components such as activities, database handlers, data models, and user interface elements. It supports complete CRUD (Create, Read, Update, Delete) operations, enabling users to add new contacts, modify existing records, remove unwanted entries, and view detailed information. Contacts are displayed using RecyclerView for better performance, and a search functionality is included to quickly locate contacts based on specific attributes like name or organization.

This application offers a structured and responsive approach to contact management while demonstrating key Android development concepts such as database integration, UI/UX design, and background processing.

Keywords*: Android Application, Contact Management System, SQLite Database, CRUD Operations, RecyclerView, Search Functionality, Mobile Application Development*

Table of Contents

Contact Management Android Applications

1. Introductions

In the modern mobile ecosystem, managing personal data efficiently is a fundamental requirement. This project features the Contact Management Android Application (V1), a utility designed to help users archive, categorize, and modify contact details directly on their Android devices. Developed as a core requirement for the Mobile Programming (CACS351) course, this initial version is built to production standards. It utilizes Google's Material Design language to offer a professional interface while ensuring data integrity through a structured local database.

2. Objectives

The primary goals for this development phase include:

1. Application Development: Creating a stable Android environment using Android Studio.
2. Data Persistence: Implementing a local SQLite database for reliable offline storage.
3. Functionality: Enabling full CRUD (Create, Read, Update, Delete) capabilities.
4. UI/UX Performance: Leveraging Recycler View and Material Card View for smooth data presentation.
5. Efficiency: Utilizing asynchronous processing to ensure the UI remains responsive during heavy database tasks.

3. Scope of Version 1

Features Included in Version 1

1. Local Contact Storage:
   All contact information is stored directly on the device using an SQLite database, allowing the app to function without internet access.
2. Complete CRUD Operations:
   The application supports full functionality to create, read, update, and delete contact records.
3. Enhanced Contact Structure:
   Each contact entry consists of the following details:

   a. First Name
   b. Last Name
   c. Phone Number
4. Improved User Interface:
   The app provides a clean and user-friendly interface using Android components such as Activities, Recycler View, Floating Action Button, and other standard UI elements.
5. Contact Detail View Screen:
   A dedicated screen is available to display complete contact information in a read-only format before making any edits.
6. Background Database Processing:
   All database-related operations are handled in background threads to ensure smooth performance and prevent UI lag.
7. Database Upgrade Handling:
   The database version has been updated to accommodate the new contact structure, ensuring proper schema migration.

Features Excluded from Version 1

- System Contact Integration:
  The application does not connect or sync with the device's built-in phone contacts.
- Live Search Functionality:
  Real-time filtering or searching of contacts by name is not included in this version.
- Caller Identification Feature:
  The app does not support identifying incoming calls based on stored contact information.
- Cloud Backup and Synchronization:
  Online storage, cloud syncing, and backup features are not implemented in this version.

4. Tools and Technologies Used

Category	Technology
Programming Language	Java
IDE	Android Studio
Platform	Android
Database	SQLite
UI Components	Activities, RecyclerView, Material Design Components
Concurrency	Java .util .concurrent.Executors
Version Control	Git
Minimum SDK	API 21

5. Application Architecture

Version 1 of the application is designed using an Activity-based architecture combined with efficient background data processing.

Architecture Flow:

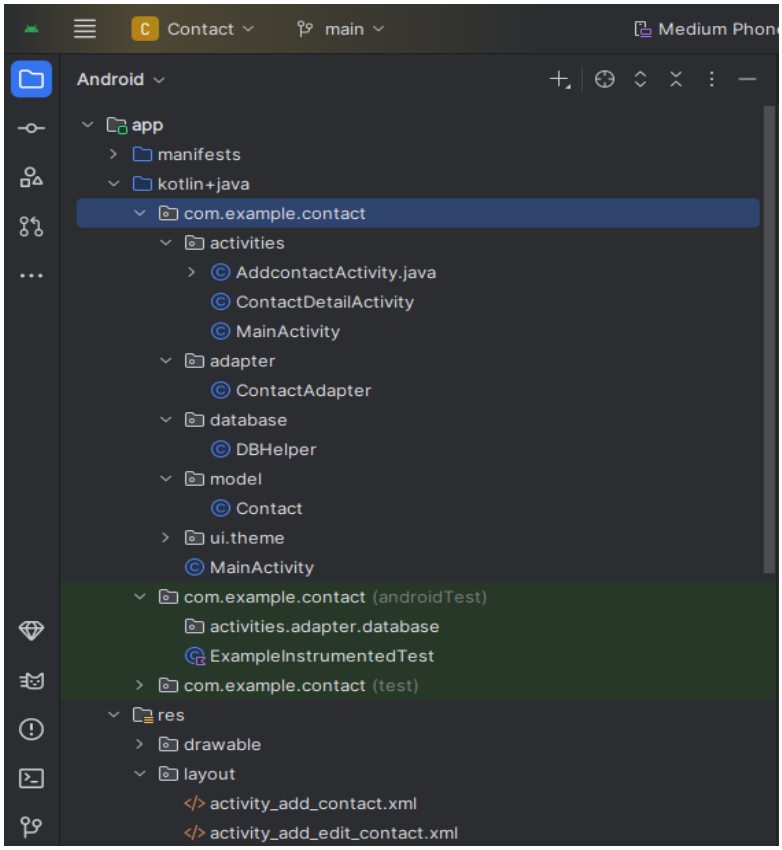
Activity → AppExecutor → SQLite Database
Activity → RecyclerView & Adapter (UI Layer)

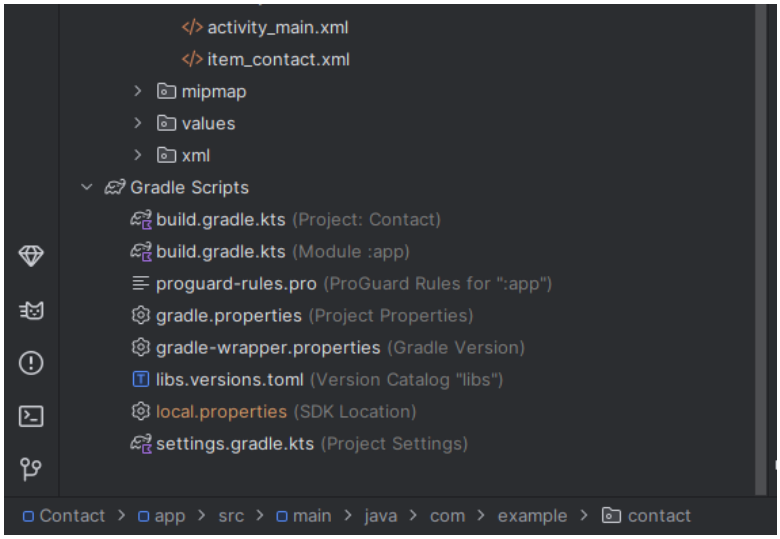
Explanation:

- **Activities:**
  Activities are responsible for handling user interactions and controlling the overall application flow.
- **AppExecutor:**
  This component manages background thread execution, ensuring that time-consuming tasks run separately from the main thread.
- **SQLite Database:**
  All contact information is stored locally using SQLite for offline accessibility.
- **RecyclerView:**
  RecyclerView is used to display the list of contacts in an efficient and scrollable format.

- **Background Processing:**
  Database operations are executed in background threads to avoid blocking the user interface.
- **UI Thread Updates:**
  Any changes to the interface are performed on the main thread to maintain stability and responsiveness.

6. Project Folder Structure





7. Git Workflow and Version Control Strategy

Version 1 of the Contact Management Application was developed using **Git** to ensure proper version control and systematic tracking of progress. Development was carried out incrementally, with changes committed regularly to reflect updates in features, user interface, and documentation.

7.1 Commit Timeline for Version 1

The development of Version 1 was completed over multiple stages, starting from the initial setup a few weeks earlier and continuing with feature and UI improvements more recently.

Initial Setup Phase (Earlier Development Stage)

1. **Initial Commit**
    - Created the base Android project.
    - Established the folder structure and added essential configuration files.
2. **README Initialization**
    - Added a basic project description and overview to the README file.

UI Enhancements and Layout Improvements (Recent Updates)

3. **Improved UI Structure**
    - Refined layout design for better alignment and usability.
4. **Spacing Adjustments**
    - Added margins to improve visual consistency and readability.
5. **Contact Item Separation**
    - Introduced visual dividers between contact entries for a cleaner look.
6. **UI Refinement**
    - Further enhanced layout components for a more polished interface.

Feature Enhancements

7. **Alphabetical Sorting Implementation**
    - Implemented sorting of contacts using SQL query:
      ORDER BY name COLLATE NOCASE ASC
    - Ensured contacts are displayed in ascending (A–Z) order regardless of case sensitivity.

Repository Synchronization

8. **Merge with Remote Repository**
    - Pulled and merged remote changes to maintain consistency between local and remote versions.

Documentation Updates

9. **README Enhancement**
    - Expanded the README file with detailed features and project explanation.
    - Finalized documentation for Version 1 submission.

This workflow reflects a structured development approach, combining early setup work with continuous improvements and proper version tracking using Git.

8. Implementation Details

8.1 Asynchronous Execution (AppExecutor)

An AppExecutor class was implemented following the Singleton design pattern to handle task execution across different threads. It utilizes the `Executors` framework to separate background operations from UI-related tasks.

- diskIO ( ):
  Handles database operations in a background thread to avoid blocking the main interface.
- mainThread ( ):
  Used to perform UI updates safely on the main thread.

This approach ensures that time-consuming operations do not affect the responsiveness of the application.

8.2 Main Activity

The MainActivity is responsible for loading and displaying contact data asynchronously.

Process:

1. A background thread retrieves the list of contacts from the SQLite database.
2. The result is passed to the main thread.
3. The RecyclerView adapter is updated with the fetched data.
4. The user interface refreshes smoothly without any lag or freezing.

8.3 Add/Edit Contact Activity

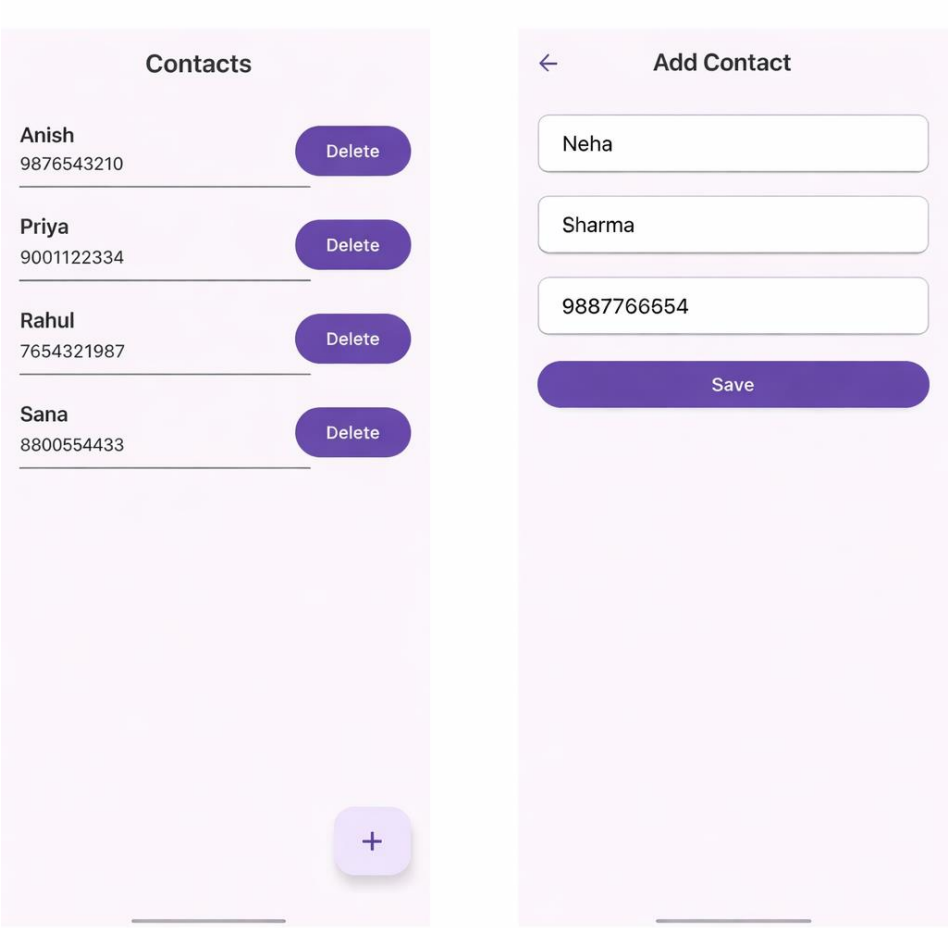
The functionality for adding and updating contacts is also handled asynchronously to maintain performance.

Process:

1. User input is validated before processing.
2. Insert or update operations are executed using diskIO() in the background.
3. Control is returned to the main thread using mainThread().
4. A success message is displayed, and the activity is closed.

This implementation ensures efficient data handling and provides a smooth user experience by properly managing background processing and UI updates.

9. Screenshots



10. Conclusion

Version 1 of the Smart Contact Management Android Application has been successfully completed and is now in a stable, usable state. The application meets all the initially defined goals, including:

- Implementation of full CRUD (Create, Read, Update, Delete) operations
- Use of an enhanced contact data model
- Development of a user-friendly interface based on Material Design principles
- Integration of asynchronous database handling

The use of background thread execution significantly improves application performance by ensuring smooth interaction without UI delays. Overall, the project reflects good Android development practices along with effective use of version control through Git.

11. Future Work

Version 2 Enhancements

- Introduce an Empty State UI when no contacts are available
- Implement real-time search/filtering by contact name
- Transition the architecture to the MVVM (Model-View-ViewModel) pattern
- Enhance the interface with improved animations and transitions

Version 3 Enhancements

- Add caller identification functionality
- Implement a spam detection system
- Enable cloud synchronization and backup
- Provide integration with the device’s system contacts